

Bankroll (CTF@CIT 2026)

Description:

Sorry! I'm out for lunch right now, but I'll check on things every few minutes! Hope nothing breaks when I'm gone!

Table of contents

- [I. Overview](#)
- [II. Source code analysis](#)
- [III. Vulnerability analysis](#)
 - [1. Information Exposure](#)
 - [2. Misconfiguration](#)
 - [3. Logic Flaw](#)
- [IV. Exploitation](#)
 - [1. Account Stealth](#)
 - [2. XSS](#)
 - [3. SQLi](#)

I. Overview

The webapp has basic functionalities such as: login, forgot-password, dashboard, note-posting service. For admin, he has an exclusive portal for checking url (enter url and get response).

The application has flaws in forgot-password service where it exposes too much information, making way for attacker to enumerate username, by that way he steals user account. In the dashboard, user can post notes contains malicious code which extracts bot cookies when it visits. Privilege escalation via admin cookie and using special dev tools for SQL injection to get the flag.

II. Source code analysis

The challenge comprises of 3 services: **main-app** (which is exposed to the Internet), **internal-app** (connected to main-app only), **bot** (invoked by some services in main-app)

```
.
├─ / [GET]
├─ /login [POST]                POST: username, password
├─ /register [GET,POST]         POST: email, password
├─ /logout [GET]
├─ /forgot-password [GET, POST]
├─ /dashboard [GET]
├─ /notes [POST]                POST: content
├─ └─ /<int:note_id> [DELETE]
└─ /internal
```

```

├── /cleanup-notes [POST]
├── /pending-check [GET]
├── /devtools [GET]
│   └── /fetch [POST]
└── POST: url

```

- `/`: show first access page with login form
- `/login`: implement as usual, hash the input password and compare with hashed password in the database. For **user**, using **SHA256**; for **admin**, using **MD5**. Logged user in and redirect to `/dashboard`
- `/logout`: nothing special
- `/forgot-password`: return error if the forgot password account doesn't exist, otherwise return sha256 hash of that account (if it is admin account then not return md5 hash)

```

main-app > .env  README.md  app.py  Preview README.md
main-app > app.py > devtools
198 @app.route('/forgot-password', methods=['POST'])
199 def forgot_password():
200     data = request.get_json()
201     username = data.get('username', '').strip()
202     conn = get_db()
203     user = conn.execute("SELECT * FROM users WHERE username = ?", (username,)).fetchone()
204     conn.close()
205     if not user:
206         return jsonify({'status': 'error', 'message': 'No account associated with that username.'}), 404
207
208     # Admin hash is never exposed
209     if user['role'] == 'admin':
210         return jsonify({
211             'status': 'ok',
212             'message': 'Account found. Recovery information has been sent to the registered email.',
213             'account': {
214                 'username': user['username'],
215                 'role': user['role']
216             }
217         })
218
219     return jsonify({
220         'status': 'ok',
221         'message': 'Account found. Recovery information has been sent.',
222         'account': {
223             'username': user['username'],
224             'role': user['role'],
225             'password_hash': user['password_hash']
226         }
227     })

```

Annotations in the code:

- `error if account doesn't exist` (points to line 206)
- `not return admin's password hash` (points to the admin role response block)
- `confirm if account exists` (points to the user role response block)
- `return user's password hash` (points to the `password_hash` field in the user role response)

- `/dashboard`: show all posts for admin, only admin posts for standard account

```

@app.route('/dashboard')
def dashboard():
    if 'user_id' not in session:
        return redirect(url_for('index'))
    conn = get_db()
    is_admin = session.get('role') == 'admin'    if account is admin, show all posts
    if is_admin:
        notes = conn.execute(
            "SELECT notes.id, notes.content, notes.created_at, users.username "
            "FROM notes JOIN users ON notes.user_id = users.id "
            "ORDER BY notes.created_at DESC"
        ).fetchall()
    else:
        notes = conn.execute(
            "SELECT notes.id, notes.content, notes.created_at, users.username "
            "FROM notes JOIN users ON notes.user_id = users.id "
            "WHERE users.username = 'svc_admin' "
            "ORDER BY notes.created_at DESC"
        ).fetchall()    if account is non-admin
                        show admin post only
    conn.close()
    return render_template('dashboard.html', username=session['username'], notes=notes, is_admin=is_admin)

```

- /notes: server does sanitize notes content before storage

The screenshot shows a code editor with the following code and annotations:

```

252
253 @app.route('/notes', methods=['POST'])
254 def add_note():
255     if 'user_id' not in session:
256         return jsonify({'status': 'error', 'message': 'Unauthorized'}), 401
257     if session.get('role') == 'admin':
258         return jsonify({'status': 'error', 'message': 'Administrative notes are read-only.'}), 403    admin cannot post notes
259     data = request.get_json() or {}
260     content = data.get('content', '').strip()
261     if not content:
262         return jsonify({'status': 'error', 'message': 'Note cannot be empty.'}), 400
263     if len(content) > 250:
264         return jsonify({'status': 'error', 'message': 'Note exceeds maximum length of 250 characters.'}), 400
265     sanitized = sanitize_note(content)
266     conn = get_db()
267     conn.execute("INSERT INTO notes (user_id, content) VALUES (?, ?)",
268                 (session['user_id'], sanitized))    sanitize notes content before putting into the database
269     conn.commit()
270     conn.close()
271     # Return success but note is NOT shown in the UI feed
272     return jsonify({'status': 'ok', 'message': 'Note submitted.'})
273
274
275 @app.route('/notes/<int:note_id>', methods=['DELETE'])
276 def delete_note(note_id):
277
278
79 def sanitize_note(content):
80     content = re.sub(r'<script.*?>.*?</script>', '', content, flags=re.IGNORECASE | re.DOTALL)
81     content = re.sub(r'<details[\\s\\S]*?>[\\s\\S]*?</details>', '', content, flags=re.IGNORECASE)
82     content = re.sub(r'<details[^>]*?>', '', content, flags=re.IGNORECASE)
83     content = re.sub(r'</details>', '', content, flags=re.IGNORECASE)
84     content = re.sub(r'javascript:', '', content, flags=re.IGNORECASE)
85     content = re.sub(r'on(load|error|click|submit|focus|toggle|begin|end|mouse\\w+|key\\w+|input|change|drag\\w+|drop|scroll|wheel|pointer\\w+)=', '', content, flags=
86     return content

```

Annotations in the image:

- admin cannot post notes (points to line 258)
- sanitize notes content before putting into the database (points to line 265)
- remove string matching appeared pattern 1 time for each (points to the sanitize_note function)

- /devtools/fetch: admin posts url, url must points to localhost at 8080.

```

main-app > app.py > devtools_fetch
319 return render_template('devtools.html', username=session['username'])
320
321
322 @app.route('/devtools/fetch', methods=['POST'])
323 def devtools_fetch():
324     if 'user_id' not in session or session.get('role') != 'admin':
325         return jsonify({'status': 'error', 'message': 'Unauthorized'}), 401
326     data = request.get_json()
327     url = data.get('url', '').strip()
328     if not url:
329         return jsonify({'status': 'error', 'message': 'No URL provided.'}), 400
330     allowed, reason = is_ssrif_allowed(url)
331     if not allowed:
332         return jsonify({'status': 'error', 'message': f'Request blocked: {reason}'})
333
334     import urllib.request
335     try:
336         with urllib.request.urlopen(url, timeout=5) as resp:
337             body = resp.read().decode('utf-8', errors='replace')
338             return jsonify({'status': 'ok', 'body': body})
339     except Exception as e:
340         return jsonify({'status': 'error', 'message': str(e)}), 500
341
342
343
344
main-app > app.py > ...
88 BLOCKED_PRIVATE = []
89 ipaddress.ip_network('10.0.0.0/8'),
90 ipaddress.ip_network('172.16.0.0/12'),
91 ipaddress.ip_network('192.168.0.0/16'),
92 ipaddress.ip_network('169.254.0.0/16'),
93
94
main-app > app.py > ...
118 return ipaddress.IPv4Address(value)
119
120
121 def is_ssrif_allowed(url):
122     try:
123         parsed = urlparse(url)
124     except Exception:
125         return False, "Invalid URL."
126
127     if parsed.scheme != 'http':
128         return False, "Only http scheme is permitted."
129
130     host = parsed.hostname or ''
131     port = parsed.port
132
133     try:
134         addr = parse_numeric_host(host)
135     except ValueError as exc:
136         return False, str(exc)
137
138     if not addr.is_loopback:
139         for net in BLOCKED_PRIVATE:
140             try:
141                 if addr in net:
142                     return False, "Access to private ranges is not permitted."
143             except TypeError:
144                 pass
145         return False, "Only loopback addresses are permitted."
146
147     if port != 8080:
148         return False, "Only port 8080 is accessible."
149     return True, "ok"
150
151
152
153 @app.route('/')
154 def index():

```

- For bot activity: it checks if exists user post and clear it for each 10 second interval. Bot logs in as admin account and delete them.

```

svc_admin_bot > bot.py > main
49 def main():
50     logging.info("Admin bot started - polling every %ds.", POLL_INTERVAL)
51     while True:
52         try:
53             if has_pending_notes():
54                 logging.info("Pending note detected - visiting dashboard.")
55                 run_bot_session()
56                 cleanup_notes()
57             except Exception as e:
58                 logging.error(f"Unhandled error: {e}")
59             time.sleep(POLL_INTERVAL)
60         def run_bot_session():
61             driver = make_driver()
62             try:
63                 driver.get(f"{BASE_URL}")
64                 wait = WebDriverWait(driver, 10)
65                 wait.until(EC.presence_of_element_located((By.ID, "username"))).send_keys(PASSWORD)
66                 driver.find_element(By.ID, "password").send_keys(PASSWORD)
67                 driver.find_element(By.CSS_SELECTOR, "button.btn").click()
68                 wait.until(EC.url_contains("/dashboard"))
69                 logging.info("Admin bot logged in and viewing dashboard.")
70                 time.sleep(5)
71             except Exception as e:
72                 logging.error(f"Bot session error: {e}")
73             finally:
74                 driver.quit()
75         def cleanup_notes():
76             try:
77                 resp = requests.post(
78                     f"{BASE_URL}/internal/cleanup-notes",
79                     headers={"X-Bot-Secret": BOT_SECRET},
80                     timeout=10)
81                 logging.info(f"Notes cleaned up: {resp.status_code}")
82             except Exception as e:
83                 logging.error(f"Cleanup failed: {e}")
84
85
svc_admin_bot > bot.py > ...
34 def has_pending_notes():
35     """Returns True only if a non-admin user has posted a note."""
36     try:
37         resp = requests.get(
38             f"{BASE_URL}/internal/pending-check",
39             headers={"X-Bot-Secret": BOT_SECRET},
40             timeout=5)
41         if resp.status_code == 200:
42             return resp.json().get('pending', False)
43
main-app > app.py > pending_check
298 @app.route('/internal/pending-check', methods=['GET'])
299 def pending_check():
300     if request.headers.get('X-Bot-Secret', '') != BOT_SECRET:
301         return jsonify({'status': 'error', 'message': 'Forbidden'}), 403
302     conn = get_db()
303     row = conn.execute(
304         "SELECT COUNT(*) as cnt FROM notes WHERE user_id NOT IN "
305         "(SELECT id FROM users WHERE username = 'svc_admin')")
306     cnt = row.fetchone()[0]
307     conn.close()
308     return jsonify({'pending': row['cnt'] > 0, 'count': row['cnt']})
309
310 @app.route('/internal/cleanup-notes', methods=['POST'])
311 def cleanup_notes():
312     if request.headers.get('X-Bot-Secret', '') != BOT_SECRET:
313         return jsonify({'status': 'error', 'message': 'Forbidden'}), 403
314     conn = get_db()
315     conn.execute(
316         "DELETE FROM notes WHERE user_id NOT IN "
317         "(SELECT id FROM users WHERE username = 'svc_admin')")
318     conn.commit()
319     conn.close()
320     return jsonify({'status': 'ok'})

```

For the `internal-app`:

There are two databases: `employees` contains data of employees, `secrets` contains one record of flag

- `/search?q=`:

```

141 @app.route('/')
142 def index():
143     return render_template_string(PAGE, query='', results=None, error=None, waf_blocked=False)
144
145
146 @app.route('/search')
147 def search():
148     q = request.args.get('q', '')
149     error = None
150     results = None
151     waf_blocked = False
152
153     if q:
154         if waf_check(q):
155             waf_blocked = True
156             return render_template_string(PAGE, query=q, results=None, error=None, waf_blocked=True)
157         try:
158             conn = get_db()
159             sql = "SELECT id, name, department, status FROM employees WHERE name = '" + q + "'"
160             cur = conn.cursor()
161             cur.execute(sql)
162             results = cur.fetchall()
163             results = [list(row) for row in results]
164             conn.close()
165         except Exception as e:
166             error = str(e)
167             results = []
168
169     return render_template_string(PAGE, query=q, results=results, error=error, waf_blocked=waf_blocked)

```

return a normal page

get search param as q

check search param q, if violate, block

concatenate user input to create SQL query and execute

There are measures preventing SQLi

```

18 def init_db():
19     conn.commit()
20     conn.close()
21
22
23 KEYWORDS = {
24     "union", "select", "from", "where",
25     "drop", "insert", "delete", "update",
26     "exec", "execute", "load_file",
27     "outfile", "dumpfile"
28 }
29
30
31 def is_bad_casing(word: str) -> bool:
32     return word.islower() or word.isupper()
33
34
35 def waf_check(value: str) -> bool:
36     for word in re.findall(r'\b\w+\b', value):
37         if word.lower() in KEYWORDS:
38             if is_bad_casing(word):
39                 return True
40
41     if "--" in value:
42         return True
43
44     return False

```

prevent SQLi

for each word in query q
if the lowercase of that word in KEYWORDS
and word is fully lowercase or uppercase
=> Ban

if there is comment sign like --
=> Ban

The main-app configs session as this

```

app.config['SESSION_COOKIE_HTTPONLY'] = False
app.config['SESSION_COOKIE_SAMESITE'] = None
app.config['SESSION_COOKIE_SECURE'] = False

```

```
app.config['PERMANENT_SESSION_LIFETIME'] = timedelta(minutes=5)
app.config['SESSION_REFRESH_EACH_REQUEST'] = False
```

which is really vulnerable to Cross-site scripting (XSS)

III. Vulnerability analysis

1. Information Exposure

In the `forgot-password` service, server informs users if the account being password-forgot exists or not, which is a venue for attacker to enumerate username via bruteforce with leaked credentials (username)

The server also returns SHA256 of standard users, which is very dangerous in case the password being used is predictable or password leaked in data breaches. Attacker would use tools to look up that password and successfully steal the account.

2. Misconfiguration

Setting `SameSite: None, HttpOnly: False, Secure: False` is like inviting XSS to steal cookie.

Besides, the bot used to clear notes is not set with `javascript disabled`. If it visits a page with stored XSS (for example in this case is XSS in notes), the cookie would be stolen easily.

3. Logic Flaw

In preventing SQLi

- In `waf_check`:

The employed database is Sqlite, it understands camel keyword like `Union`, `uNiOn`, etc. not just `union` and `UNION` as dev thinks. So he may miss this functionality of SQLite

```
if word.lower() in KEYWORDS:
    if is_bad_casing(word):
        return True
```

In this case, instead of return True immediately upon finding the forbidden keyword, dev still checks the case of `original word` is bad or not ? Which leaves way for attacker to bypass the filter as above.

- In `Buidling query from q`:

The dev concatenates `q` from untrusted data (user) into the query directly (though there is filter but it's bypassed now =))) rather than employs famous measures such as: prepared statements with parameterized queries or allow-list.

In preventing XSS

The server does have sanitization mechanism which removes forbidden keyword (which is believed to be dangerous) but not recursively.

For example: `I'm<script> bbt` will be `I'm bbt` However, if payload is `<script><script></script></script>` will be `<script>`, and it is clean (at least in the point of view of server =))).

So using nested payload will bypass this filter.

IV. Exploitation

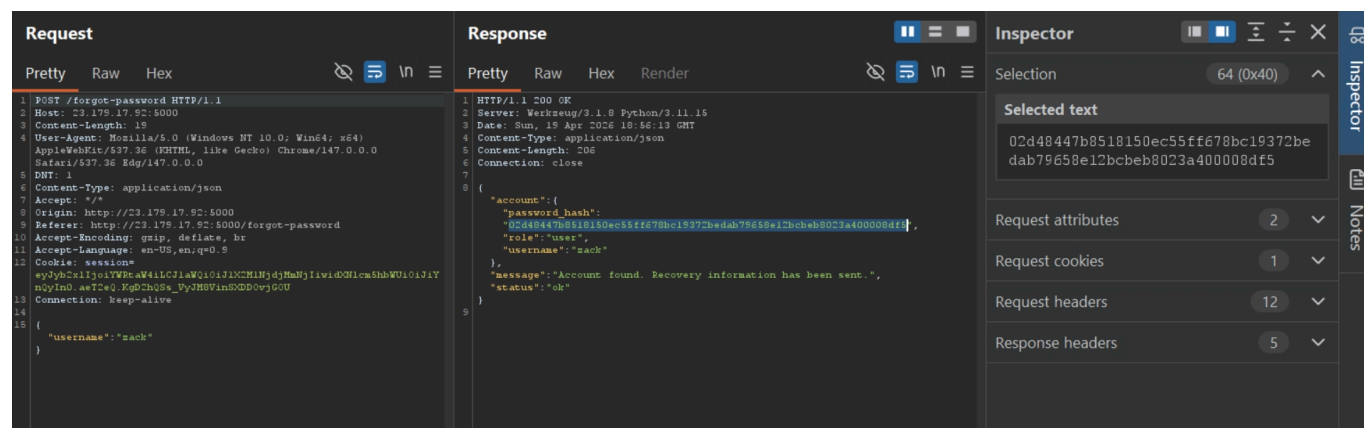
The main idea here is:

- **Firstly:** steal the account via bruteforce username and crack hashed password
- **Secondly:** login and post note with XSS payload stealing admin session
- **Finally:** Login as admin and exploit SQLi getting flag

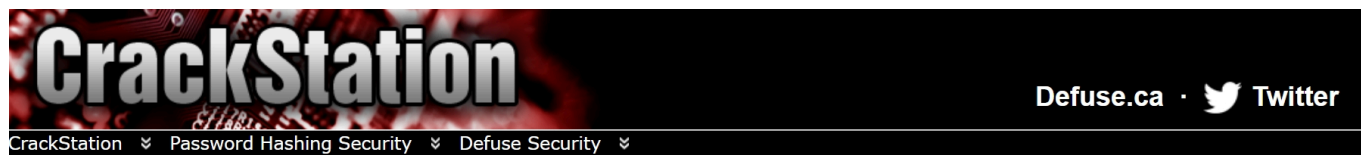
1. Account Stealth

Since we don't have any credentials or account, we must abuse forgot-password service for username enumeration.

Using Burp Intruder with wordlists from `seclists/Usernames/Names/names.txt` from kali, I found `zack`

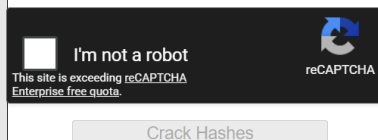


Got the hashed password of `zack`. I bring it to crackstation and wish for lucky.



Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:



Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

Hash	Type	Result
02d48447b8518150ec55ff678bc19372bedab79658e12bcbeb8023a400008df5	sha256	ryLis@1024

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

[Download CrackStation's Wordlist](#)

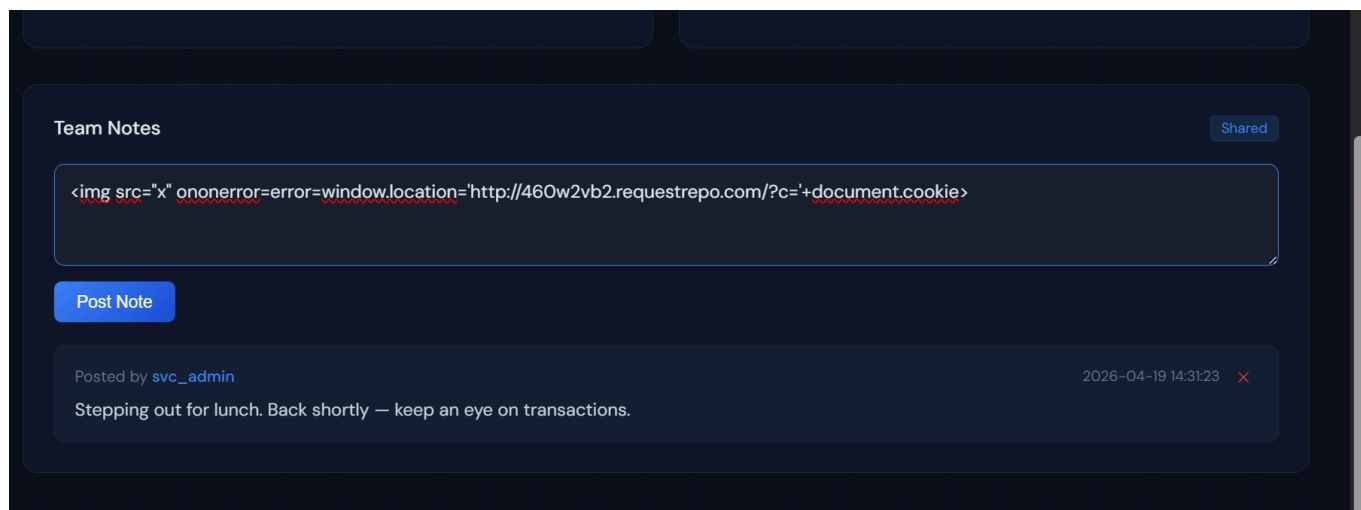
Yeah, the password is ryLis@1024

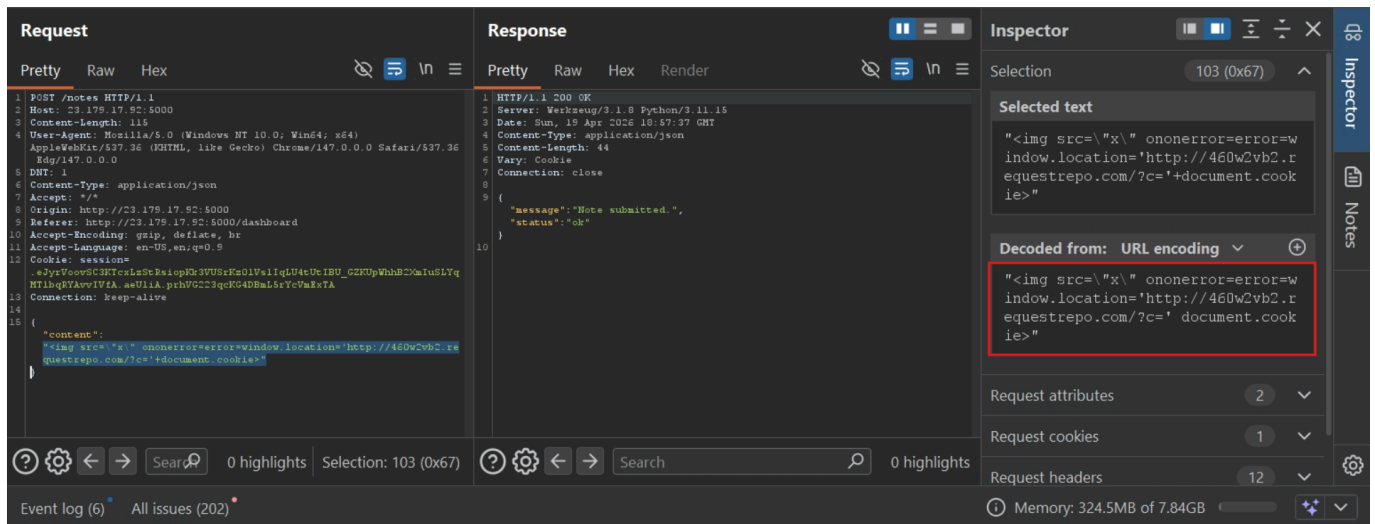
So the credentials is zack:ryLis@1024. Let's login.

2. XSS to steal Admin session from Bot

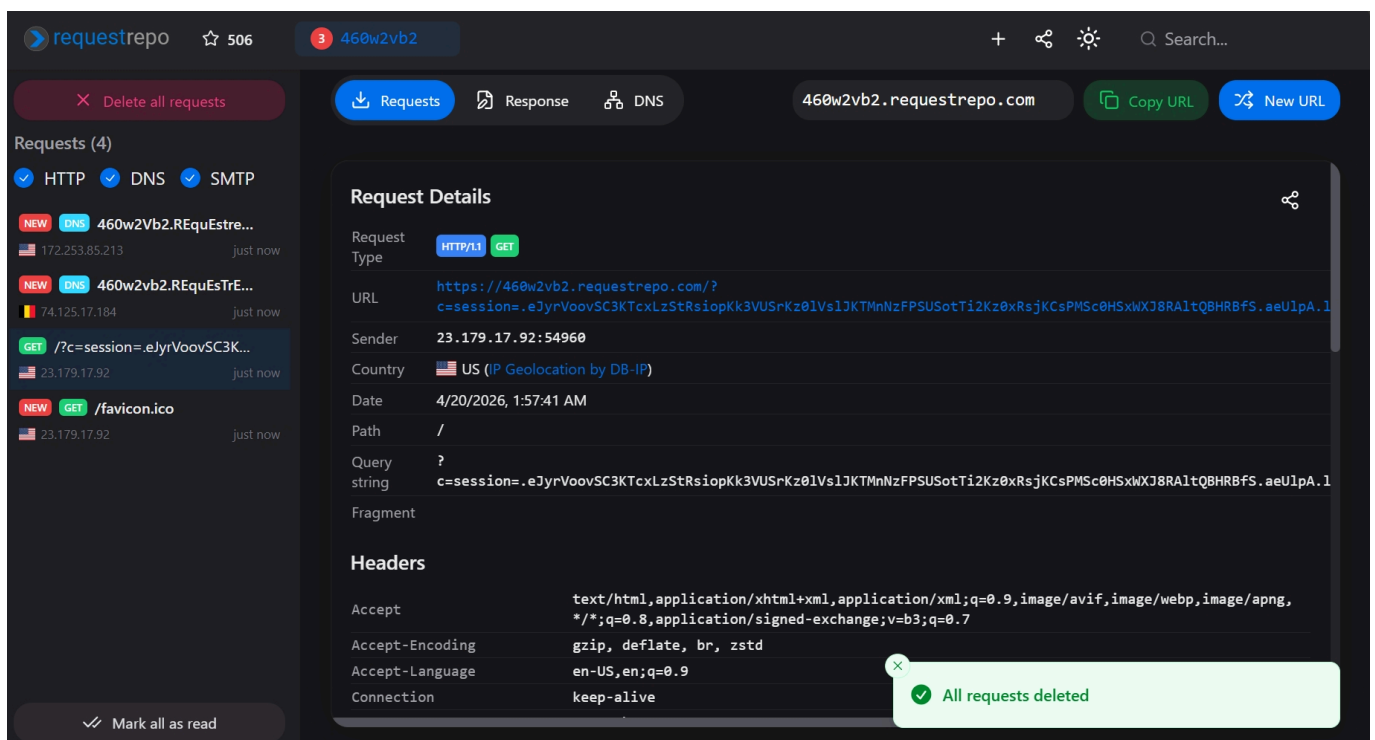
Since bot logs in as admin and visits /dashboard in which all posts of every one is shown. One of them will contain our note with XSS payload (that has been modified to bypass the XSS filter as I mentioned above)

Payload:





Wait for 10s for the bot to visit our note. Check at our webhook



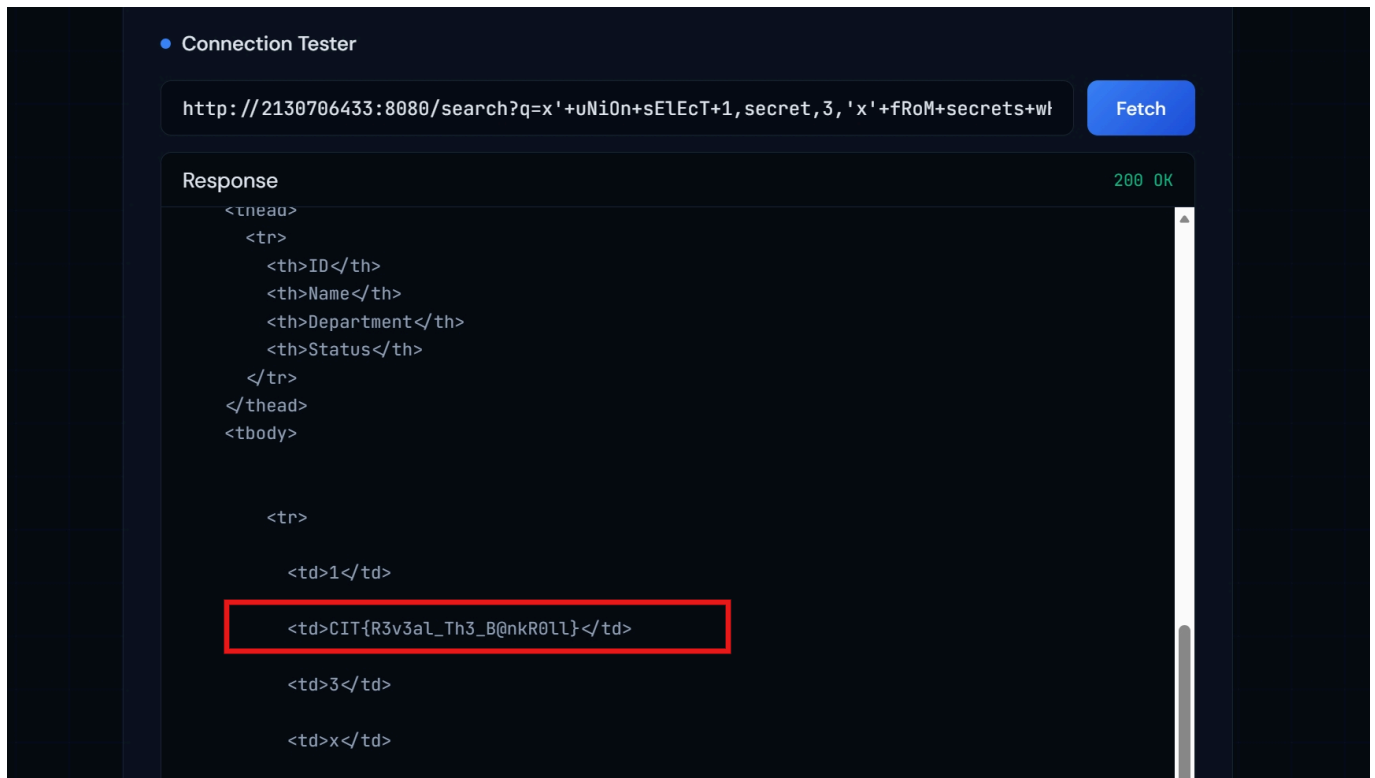
Got the session! Replace current session with the stolen one to become admin.

3. SQL Injection

Head to `/devtools/fetch`

Exploiting this query is now easy, I use **UNION** technique.

However when I try `#` to comment the last quote `'`, it doesn't work eventhough I have encode the `#` into `%23`



I use `127.0.0.1:8080` because the server requires it to be. In the picture, the number in netloc is in form of Dotted Decimal (decimal IP). And server still understands it via `parse_numeric_host`.

Flag: **CIT{R3v3a1_Th3_B@nkR0ll}**